

Tarea 2

Integrantes:

- Sofía Salinas Rico - ssalinas@unal.edu.co
- Camilo Chitivo Cerinza - cchitivo@unal.edu.co

Considere un tablero de ajedrez estándar de 8×8 .

Las columnas del tablero se identifican mediante las letras a, b, c, d, e, f, g, h

y las filas mediante los números $1, 2, \dots, 8$.

Por tanto, cada casilla se identifica mediante una letra y un número. El tablero queda identificado de la siguiente manera:

8	a8	b8	c8	d8	e8	f8	g8	h8
7	a7	b7	c7	d7	e7	f7	g7	h7
6	a6	b6	c6	d6	e6	f6	g6	h6
5	a5	b5	c5	d5	e5	f5	g5	h5
4	a4	b4	c4	d4	e4	f4	g4	h4
3	a3	b3	c3	d3	e3	f3	g3	h3
2	a2	b2	c2	d2	e2	f2	g2	h2
1	a1	b1	c1	d1	e1	f1	g1	h1
	a	b	c	d	e	f	g	h

Considere un caballo que se desplaza aleatoriamente sobre el tablero. Suponga que no hay otras piezas. El caballo realiza los movimientos usuales del ajedrez, se mueve formando una letra "L" mayúscula: avanza dos casillas en línea recta (en dirección vertical u horizontal) y luego da un giro de una casilla hacia un lado en ángulo recto. En cada movimiento, el caballo elige uniformemente al azar una de las casillas a las que puede desplazarse legalmente desde su posición actual.

Sea X_n = posición del caballo en el n -ésimo movimiento, $n = 0, 1, 2, \dots$

De esta manera, $X = (X_n)_{n \geq 0}$ es una Cadena de Markov.

Ejercicios

1. Construcción de la cadena de Markov.

a) Describa el espacio de estados S del proceso $X = (X_n)_{n \geq 0}$. ¿Cuántos estados tiene?

b) Para un estado $x \in S$, defina

$N(x) = \{y \in S : \text{el caballo puede pasar de } x \text{ a } y \text{ en un solo movimiento}\}$

$$|N(x)| = \text{cardinal}(N(x))$$

Determine $N(x)$ y $|N(x)|$ para cada $x \in S$. Presente los resultados en una tabla que contenga x , $N(x)$, $|N(x)|$.

c) Teniendo en cuenta que el caballo elige moverse uniformemente entre todos los movimientos legales disponibles desde x , calcule las probabilidades de transición:

$$P_{xy} = P(X_{n+1} = y \mid X_n = x) = \begin{cases} \frac{1}{|N(x)|}, & y \in N(x) \\ 0, & y \notin N(x) \end{cases}$$

Ejemplo:

$$P_{a1,y} = \begin{cases} \frac{1}{2}, & y = b3, c2 \\ 0, & \text{caso contrario} \end{cases}$$

d) Presente las probabilidades de transición $P_{h8,y}, P_{g1,y}, P_{b7,y}, P_{f7,y}, P_{e5,y}$.

e) Fije un orden para las 64 casillas del tablero. Por ejemplo, puede utilizar:

$$a1, a2, \dots, a8, b1, b2, \dots, b8, \dots, h1, \dots, h8$$

Construya la matriz de transición $P = (P_{xy})_{x,y \in S}$.

La matriz puede ser construida utilizando un programa, incluya en el informe el código utilizado (o el enlace al código usado). No es necesario escribir todas sus entradas. Incluya en el informe, al menos, las filas de la matriz correspondientes a las casillas $g1$ y $h6$.

f) ¿La matriz P es simétrica? Justifique su respuesta.

Solución

a)

Tradicionalmente los caballos inician en $b1, g1, b8$ y $g8$, supongamos que $X_0 = b1$, luego el espacio de estados S se define como parejas x, y tales que $x \in \{a, b, c, d, e, f, h\}$ y $1 < y \leq 8$.

Para saber cuantos estados tenemos hagamos una función que recorre el tablero.

```
In [1]: def posibles_vecinos(x,y):  
    return [(x+1,y+2),(x+1,y-2),(x-1,y+2),(x-1,y-2),(x+2,y+1),(x+2,y-1),(x-2,y+1),(x-2,y-1)]  
def vecinos_validos(x,y):  
    return [(i,j) for (i,j) in posibles_vecinos(x,y) if i>=1 and i<=8 and j>=1 and j<=8]
```

Vamos a utilizar dos approach para este proyecto, uno iterativo por cada una de las casillas y otro recursivo.

```
In [2]: def todos_los_vecinos_validos_iterativo():  
        return { (i,j):vecinos_validos(i,j) for i in range(1,9) for j in range(1,9) }
```

```
In [3]: todos_los_vecinos_iterativo = todos_los_vecinos_validos_iterativo()  
print(todos_los_vecinos_iterativo)
```

```
{(1, 1): [(2, 3), (3, 2)], (1, 2): [(2, 4), (3, 3), (3, 1)], (1, 3): [(2, 5), (2, 1), (3, 4), (3, 2)], (1, 4): [(2, 6), (2, 2), (3, 5), (3, 3)],  
(1, 5): [(2, 7), (2, 3), (3, 6), (3, 4)], (1, 6): [(2, 8), (2, 4), (3, 7), (3, 5)], (1, 7): [(2, 5), (3, 8), (3, 6)], (1, 8): [(2, 6), (3, 7)], (2,  
1): [(3, 3), (1, 3), (4, 2)], (2, 2): [(3, 4), (1, 4), (4, 3), (4, 1)], (2, 3): [(3, 5), (3, 1), (1, 5), (1, 1), (4, 4), (4, 2)], (2, 4): [(3, 6),  
(3, 2), (1, 6), (1, 2), (4, 5), (4, 3)], (2, 5): [(3, 7), (3, 3), (1, 7), (1, 3), (4, 6), (4, 4)], (2, 6): [(3, 8), (3, 4), (1, 8), (1, 4), (4, 7),  
(4, 5)], (2, 7): [(3, 5), (1, 5), (4, 8), (4, 6)], (2, 8): [(3, 6), (1, 6), (4, 7)], (3, 1): [(4, 3), (2, 3), (5, 2), (1, 2)], (3, 2): [(4, 4), (2,  
4), (5, 3), (5, 1), (1, 3), (1, 1)], (3, 3): [(4, 5), (4, 1), (2, 5), (2, 1), (5, 4), (5, 2), (1, 4), (1, 2)], (3, 4): [(4, 6), (4, 2), (2, 6), (2,  
2), (5, 5), (5, 3), (1, 5), (1, 3)], (3, 5): [(4, 7), (4, 3), (2, 7), (2, 3), (5, 6), (5, 4), (1, 6), (1, 4)], (3, 6): [(4, 8), (4, 4), (2, 8), (2,  
4), (5, 7), (5, 5), (1, 7), (1, 5)], (3, 7): [(4, 5), (2, 5), (5, 8), (5, 6), (1, 8), (1, 6)], (3, 8): [(4, 6), (2, 6), (5, 7), (1, 7)], (4, 1):  
[(5, 3), (3, 3), (6, 2), (2, 2)], (4, 2): [(5, 4), (3, 4), (6, 3), (6, 1), (2, 3), (2, 1)], (4, 3): [(5, 5), (5, 1), (3, 5), (3, 1), (6, 4), (6,  
2), (2, 4), (2, 2)], (4, 4): [(5, 6), (5, 2), (3, 6), (3, 2), (6, 5), (6, 3), (2, 5), (2, 3)], (4, 5): [(5, 7), (5, 3), (3, 7), (3, 3), (6, 6), (6,  
4), (2, 6), (2, 4)], (4, 6): [(5, 8), (5, 4), (3, 8), (3, 4), (6, 7), (6, 5), (2, 7), (2, 5)], (4, 7): [(5, 5), (3, 5), (6, 8), (6, 6), (2, 8), (2,  
6)], (4, 8): [(5, 6), (3, 6), (6, 7), (2, 7)], (5, 1): [(6, 3), (4, 3), (7, 2), (3, 2)], (5, 2): [(6, 4), (4, 4), (7, 3), (7, 1), (3, 3), (3, 1)],  
(5, 3): [(6, 5), (6, 1), (4, 5), (4, 1), (7, 4), (7, 2), (3, 4), (3, 2)], (5, 4): [(6, 6), (6, 2), (4, 6), (4, 2), (7, 5), (7, 3), (3, 5), (3, 3)],  
(5, 5): [(6, 7), (6, 3), (4, 7), (4, 3), (7, 6), (7, 4), (3, 6), (3, 4)], (5, 6): [(6, 8), (6, 4), (4, 8), (4, 4), (7, 7), (7, 5), (3, 7), (3, 5)],  
(5, 7): [(6, 5), (4, 5), (7, 8), (7, 6), (3, 8), (3, 6)], (5, 8): [(6, 6), (4, 6), (7, 7), (3, 7)], (6, 1): [(7, 3), (5, 3), (8, 2), (4, 2)], (6,  
2): [(7, 4), (5, 4), (8, 3), (8, 1), (4, 3), (4, 1)], (6, 3): [(7, 5), (7, 1), (5, 5), (5, 1), (8, 4), (8, 2), (4, 4), (4, 2)], (6, 4): [(7, 6),  
(7, 2), (5, 6), (5, 2), (8, 5), (8, 3), (4, 5), (4, 3)], (6, 5): [(7, 7), (7, 3), (5, 7), (5, 3), (8, 6), (8, 4), (4, 6), (4, 4)], (6, 6): [(7, 8),  
(7, 4), (5, 8), (5, 4), (8, 7), (8, 5), (4, 7), (4, 5)], (6, 7): [(7, 5), (5, 5), (8, 8), (8, 6), (4, 8), (4, 6)], (6, 8): [(7, 6), (5, 6), (8, 7),  
(4, 7)], (7, 1): [(8, 3), (6, 3), (5, 2)], (7, 2): [(8, 4), (6, 4), (5, 3), (5, 1)], (7, 3): [(8, 5), (8, 1), (6, 5), (6, 1), (5, 4), (5, 2)], (7,  
4): [(8, 6), (8, 2), (6, 6), (6, 2), (5, 5), (5, 3)], (7, 5): [(8, 7), (8, 3), (6, 7), (6, 3), (5, 6), (5, 4)], (7, 6): [(8, 8), (8, 4), (6, 8),  
(6, 4), (5, 7), (5, 5)], (7, 7): [(8, 5), (6, 5), (5, 8), (5, 6)], (7, 8): [(8, 6), (6, 6), (5, 7)], (8, 1): [(7, 3), (6, 2)], (8, 2): [(7, 4), (6,  
3), (6, 1)], (8, 3): [(7, 5), (7, 1), (6, 4), (6, 2)], (8, 4): [(7, 6), (7, 2), (6, 5), (6, 3)], (8, 5): [(7, 7), (7, 3), (6, 6), (6, 4)], (8, 6):  
[(7, 8), (7, 4), (6, 7), (6, 5)], (8, 7): [(7, 5), (6, 8), (6, 6)], (8, 8): [(7, 6), (6, 7)]}
```

```
In [4]: inicial = (2,1)  
todos_los_vecinos_recursivos = {}  
def todos_los_vecinos_validos_recursivo(x,y):  
    nuevos_vecinos = vecinos_validos(x,y)  
    todos_los_vecinos_recursivos[(x,y)] = nuevos_vecinos  
    for vecino in nuevos_vecinos:  
        if vecino not in todos_los_vecinos_recursivos:  
            todos_los_vecinos_validos_recursivo(vecino[0],vecino[1])
```

```
In [5]: todos_los_vecinos_validos_recursivo(inicial[0],inicial[1])  
print(todos_los_vecinos_recursivos)
```

```
{(2, 1): [(3, 3), (1, 3), (4, 2)], (3, 3): [(4, 5), (4, 1), (2, 5), (2, 1), (5, 4), (5, 2), (1, 4), (1, 2)], (4, 5): [(5, 7), (5, 3), (3, 7), (3, 3), (6, 6), (6, 4), (2, 6), (2, 4)], (5, 7): [(6, 5), (4, 5), (7, 8), (7, 6), (3, 8), (3, 6)], (6, 5): [(7, 7), (7, 3), (5, 7), (5, 3), (8, 6), (8, 4), (4, 6), (4, 4)], (7, 7): [(8, 5), (6, 5), (5, 8), (5, 6)], (8, 5): [(7, 7), (7, 3), (6, 6), (6, 4)], (7, 3): [(8, 5), (8, 1), (6, 5), (6, 1), (5, 4), (5, 2)], (8, 1): [(7, 3), (6, 2)], (6, 2): [(7, 4), (5, 4), (8, 3), (8, 1), (4, 3), (4, 1)], (7, 4): [(8, 6), (8, 2), (6, 6), (6, 2), (5, 5), (5, 3)], (8, 6): [(7, 8), (7, 4), (6, 7), (6, 5)], (7, 8): [(8, 6), (6, 6), (5, 7)], (6, 6): [(7, 8), (7, 4), (5, 8), (5, 4), (8, 7), (8, 5), (4, 7), (4, 5)], (5, 8): [(6, 6), (4, 6), (7, 7), (3, 7)], (4, 6): [(5, 8), (5, 4), (3, 8), (3, 4), (6, 7), (6, 5), (2, 7), (2, 5)], (5, 4): [(6, 6), (6, 2), (4, 6), (4, 2), (7, 5), (7, 3), (3, 5), (3, 3)], (4, 2): [(5, 4), (3, 4), (6, 3), (6, 1), (2, 3), (2, 1)], (3, 4): [(4, 6), (4, 2), (2, 6), (2, 2), (5, 5), (5, 3), (1, 5), (1, 3)], (2, 6): [(3, 8), (3, 4), (1, 8), (1, 4), (4, 7), (4, 5)], (3, 8): [(4, 6), (2, 6), (5, 7), (1, 7)], (1, 7): [(2, 5), (3, 8), (3, 6)], (2, 5): [(3, 7), (3, 3), (1, 7), (1, 3), (4, 6), (4, 4)], (3, 7): [(4, 5), (2, 5), (5, 8), (5, 6), (1, 8), (1, 6)], (5, 6): [(6, 8), (6, 4), (4, 8), (4, 4), (7, 7), (7, 5), (3, 7), (3, 5)], (6, 8): [(7, 6), (5, 6), (8, 7), (4, 7)], (7, 6): [(8, 8), (8, 4), (6, 8), (6, 4), (5, 7), (5, 5)], (8, 8): [(7, 6), (6, 7)], (6, 7): [(7, 5), (5, 5), (8, 8), (8, 6), (4, 8), (4, 6)], (7, 5): [(8, 7), (8, 3), (6, 7), (6, 3), (5, 6), (5, 4)], (8, 7): [(7, 5), (6, 8), (6, 6)], (8, 3): [(7, 5), (7, 1), (6, 4), (6, 2)], (7, 1): [(8, 3), (6, 3), (5, 2)], (6, 3): [(7, 5), (7, 1), (5, 5), (5, 1), (8, 4), (8, 2), (4, 4), (4, 2)], (5, 5): [(6, 7), (6, 3), (4, 7), (4, 3), (7, 6), (7, 4), (3, 6), (3, 4)], (4, 7): [(5, 5), (3, 5), (6, 8), (6, 6), (2, 8), (2, 6)], (3, 5): [(4, 7), (4, 3), (2, 7), (2, 3), (5, 6), (5, 4), (1, 6), (1, 4)], (4, 3): [(5, 5), (5, 1), (3, 5), (3, 1), (6, 4), (6, 2), (2, 4), (2, 2)], (5, 1): [(6, 3), (4, 3), (7, 2), (3, 2)], (7, 2): [(8, 4), (6, 4), (5, 3), (5, 1)], (8, 4): [(7, 6), (7, 2), (6, 5), (6, 3)], (6, 4): [(7, 6), (7, 2), (5, 6), (5, 2), (8, 5), (8, 3), (4, 5), (4, 3)], (5, 2): [(6, 4), (4, 4), (7, 3), (7, 1), (3, 3), (3, 1)], (4, 4): [(5, 6), (5, 2), (3, 6), (3, 2), (6, 5), (6, 3), (2, 5), (2, 3)], (3, 6): [(4, 8), (4, 4), (2, 8), (2, 4), (5, 7), (5, 5), (1, 7), (1, 5)], (4, 8): [(5, 6), (3, 6), (6, 7), (2, 7)], (2, 7): [(3, 5), (1, 5), (4, 8), (4, 6)], (1, 5): [(2, 7), (2, 3), (3, 6), (3, 4)], (2, 3): [(3, 5), (3, 1), (1, 5), (1, 1), (4, 4), (4, 2)], (3, 1): [(4, 3), (2, 3), (5, 2), (1, 2)], (1, 2): [(2, 4), (3, 3), (3, 1)], (2, 4): [(3, 6), (3, 2), (1, 6), (1, 2), (4, 5), (4, 3)], (3, 2): [(4, 4), (2, 4), (5, 3), (5, 1), (1, 3), (1, 1)], (5, 3): [(6, 5), (6, 1), (4, 5), (4, 1), (7, 4), (7, 2), (3, 4), (3, 2)], (6, 1): [(7, 3), (5, 3), (8, 2), (4, 2)], (8, 2): [(7, 4), (6, 3), (6, 1)], (4, 1): [(5, 3), (3, 3), (6, 2), (2, 2)], (2, 2): [(3, 4), (1, 4), (4, 3), (4, 1)], (1, 4): [(2, 6), (2, 2), (3, 5), (3, 3)], (1, 3): [(2, 5), (2, 1), (3, 4), (3, 2)], (1, 1): [(2, 3), (3, 2)], (1, 6): [(2, 8), (2, 4), (3, 7), (3, 5)], (2, 8): [(3, 6), (1, 6), (4, 7)], (1, 8): [(2, 6), (3, 7)]}
```

como vamos a confiar en los recursivos, veamos cuantos vecinos totales obtuvimos

```
In [6]: print(len(todos_los_vecinos_recursivos.keys()))
```

64

Entonces todas las posiciones del tablero de ajedrez se pueden alcanzar desde la posición inicial (2,1) usando movimientos de caballo.

b)

Dado que ya tenemos estos resultados, podemos generar la tabla.

```
In [7]: map_letras = {1:'a', 2:'b', 3:'c', 4:'d', 5:'e', 6:'f', 7:'g', 8:'h'}
lambda_map = lambda x: (map_letras[x[0]]+str(x[1]))
resultados_crudos = [(x, N_x, len(N_x)) for x, N_x in todos_los_vecinos_iterativo.items()]
resultados = [(lambda_map(x), [lambda_map(y) for y in N_x], len(N_x)) for x, N_x, len_N_x in resultados_crudos]
```

```
In [8]: from tabulate import tabulate

print(tabulate(resultados, headers=["x", "N(x)", "|N(x)|"], tablefmt="github") )
```

x	N(x)	
a1	['b3', 'c2']	2
a2	['b4', 'c3', 'c1']	3
a3	['b5', 'b1', 'c4', 'c2']	4
a4	['b6', 'b2', 'c5', 'c3']	4
a5	['b7', 'b3', 'c6', 'c4']	4
a6	['b8', 'b4', 'c7', 'c5']	4
a7	['b5', 'c8', 'c6']	3
a8	['b6', 'c7']	2
b1	['c3', 'a3', 'd2']	3
b2	['c4', 'a4', 'd3', 'd1']	4
b3	['c5', 'c1', 'a5', 'a1', 'd4', 'd2']	6
b4	['c6', 'c2', 'a6', 'a2', 'd5', 'd3']	6
b5	['c7', 'c3', 'a7', 'a3', 'd6', 'd4']	6
b6	['c8', 'c4', 'a8', 'a4', 'd7', 'd5']	6
b7	['c5', 'a5', 'd8', 'd6']	4
b8	['c6', 'a6', 'd7']	3
c1	['d3', 'b3', 'e2', 'a2']	4
c2	['d4', 'b4', 'e3', 'e1', 'a3', 'a1']	6
c3	['d5', 'd1', 'b5', 'b1', 'e4', 'e2', 'a4', 'a2']	8
c4	['d6', 'd2', 'b6', 'b2', 'e5', 'e3', 'a5', 'a3']	8
c5	['d7', 'd3', 'b7', 'b3', 'e6', 'e4', 'a6', 'a4']	8
c6	['d8', 'd4', 'b8', 'b4', 'e7', 'e5', 'a7', 'a5']	8
c7	['d5', 'b5', 'e8', 'e6', 'a8', 'a6']	6
c8	['d6', 'b6', 'e7', 'a7']	4
d1	['e3', 'c3', 'f2', 'b2']	4
d2	['e4', 'c4', 'f3', 'f1', 'b3', 'b1']	6
d3	['e5', 'e1', 'c5', 'c1', 'f4', 'f2', 'b4', 'b2']	8
d4	['e6', 'e2', 'c6', 'c2', 'f5', 'f3', 'b5', 'b3']	8
d5	['e7', 'e3', 'c7', 'c3', 'f6', 'f4', 'b6', 'b4']	8
d6	['e8', 'e4', 'c8', 'c4', 'f7', 'f5', 'b7', 'b5']	8
d7	['e5', 'c5', 'f8', 'f6', 'b8', 'b6']	6
d8	['e6', 'c6', 'f7', 'b7']	4
e1	['f3', 'd3', 'g2', 'c2']	4
e2	['f4', 'd4', 'g3', 'g1', 'c3', 'c1']	6
e3	['f5', 'f1', 'd5', 'd1', 'g4', 'g2', 'c4', 'c2']	8
e4	['f6', 'f2', 'd6', 'd2', 'g5', 'g3', 'c5', 'c3']	8
e5	['f7', 'f3', 'd7', 'd3', 'g6', 'g4', 'c6', 'c4']	8
e6	['f8', 'f4', 'd8', 'd4', 'g7', 'g5', 'c7', 'c5']	8
e7	['f5', 'd5', 'g8', 'g6', 'c8', 'c6']	6
e8	['f6', 'd6', 'g7', 'c7']	4
f1	['g3', 'e3', 'h2', 'd2']	4
f2	['g4', 'e4', 'h3', 'h1', 'd3', 'd1']	6
f3	['g5', 'g1', 'e5', 'e1', 'h4', 'h2', 'd4', 'd2']	8
f4	['g6', 'g2', 'e6', 'e2', 'h5', 'h3', 'd5', 'd3']	8
f5	['g7', 'g3', 'e7', 'e3', 'h6', 'h4', 'd6', 'd4']	8
f6	['g8', 'g4', 'e8', 'e4', 'h7', 'h5', 'd7', 'd5']	8
f7	['g5', 'e5', 'h8', 'h6', 'd8', 'd6']	6
f8	['g6', 'e6', 'h7', 'd7']	4
g1	['h3', 'f3', 'e2']	3
g2	['h4', 'f4', 'e3', 'e1']	4
g3	['h5', 'h1', 'f5', 'f1', 'e4', 'e2']	6

g4	['h6', 'h2', 'f6', 'f2', 'e5', 'e3']	6
g5	['h7', 'h3', 'f7', 'f3', 'e6', 'e4']	6
g6	['h8', 'h4', 'f8', 'f4', 'e7', 'e5']	6
g7	['h5', 'f5', 'e8', 'e6']	4
g8	['h6', 'f6', 'e7']	3
h1	['g3', 'f2']	2
h2	['g4', 'f3', 'f1']	3
h3	['g5', 'g1', 'f4', 'f2']	4
h4	['g6', 'g2', 'f5', 'f3']	4
h5	['g7', 'g3', 'f6', 'f4']	4
h6	['g8', 'g4', 'f7', 'f5']	4
h7	['g5', 'f8', 'f6']	3
h8	['g6', 'f7']	2

c)

Para responder los problemas c) y d) construimos un generador de código `tex`.

```
In [9]: from IPython.display import Math, Latex

def probabilidad_de_transicion(x, N_x, len_N_x):
    elementos_y = ", ".join(N_x)

    # Construimos la cadena de LaTeX
    latex_code = (
        rf"P_{{{x},{y}}} = \begin{{cases}} \frac{{1}}{{{len_N_x}}}, & y ="
        rf"{elementos_y} \\ 0, & \text{{caso contrario}} \end{{cases}}"
    )

    display(Latex(rf"${latex_code}$"))
```

```
In [10]: probabilidad_de_transicion(resultados[0][0], resultados[0][1], resultados[0][2])
```

$$P_{a1,y} = \begin{cases} \frac{1}{2}, & y = b3, c2 \\ 0, & \text{caso contrario} \end{cases}$$

```
In [11]: for x, N_x, len_N_x in resultados:
    probabilidad_de_transicion(x, N_x, len_N_x)
```

$$P_{a1,y} = \begin{cases} \frac{1}{2}, & y = b3, c2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{a2,y} = \begin{cases} \frac{1}{3}, & y = b4, c3, c1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{a3,y} = \begin{cases} \frac{1}{4}, & y = b5, b1, c4, c2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{a4,y} = \begin{cases} \frac{1}{4}, & y = b6, b2, c5, c3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{a5,y} = \begin{cases} \frac{1}{4}, & y = b7, b3, c6, c4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{a6,y} = \begin{cases} \frac{1}{4}, & y = b8, b4, c7, c5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{a7,y} = \begin{cases} \frac{1}{3}, & y = b5, c8, c6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{a8,y} = \begin{cases} \frac{1}{2}, & y = b6, c7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b1,y} = \begin{cases} \frac{1}{3}, & y = c3, a3, d2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b2,y} = \begin{cases} \frac{1}{4}, & y = c4, a4, d3, d1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b3,y} = \begin{cases} \frac{1}{6}, & y = c5, c1, a5, a1, d4, d2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b4,y} = \begin{cases} \frac{1}{6}, & y = c6, c2, a6, a2, d5, d3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b5,y} = \begin{cases} \frac{1}{6}, & y = c7, c3, a7, a3, d6, d4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b6,y} = \begin{cases} \frac{1}{6}, & y = c8, c4, a8, a4, d7, d5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b7,y} = \begin{cases} \frac{1}{4}, & y = c5, a5, d8, d6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b8,y} = \begin{cases} \frac{1}{3}, & y = c6, a6, d7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c1,y} = \begin{cases} \frac{1}{4}, & y = d3, b3, e2, a2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c2,y} = \begin{cases} \frac{1}{6}, & y = d4, b4, e3, e1, a3, a1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c3,y} = \begin{cases} \frac{1}{8}, & y = d5, d1, b5, b1, e4, e2, a4, a2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c4,y} = \begin{cases} \frac{1}{8}, & y = d6, d2, b6, b2, e5, e3, a5, a3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c5,y} = \begin{cases} \frac{1}{8}, & y = d7, d3, b7, b3, e6, e4, a6, a4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c6,y} = \begin{cases} \frac{1}{8}, & y = d8, d4, b8, b4, e7, e5, a7, a5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c7,y} = \begin{cases} \frac{1}{6}, & y = d5, b5, e8, e6, a8, a6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{c8,y} = \begin{cases} \frac{1}{4}, & y = d6, b6, e7, a7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d1,y} = \begin{cases} \frac{1}{4}, & y = e3, c3, f2, b2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d2,y} = \begin{cases} \frac{1}{6}, & y = e4, c4, f3, f1, b3, b1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d3,y} = \begin{cases} \frac{1}{8}, & y = e5, e1, c5, c1, f4, f2, b4, b2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d4,y} = \begin{cases} \frac{1}{8}, & y = e6, e2, c6, c2, f5, f3, b5, b3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d5,y} = \begin{cases} \frac{1}{8}, & y = e7, e3, c7, c3, f6, f4, b6, b4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d6,y} = \begin{cases} \frac{1}{8}, & y = e8, e4, c8, c4, f7, f5, b7, b5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d7,y} = \begin{cases} \frac{1}{6}, & y = e5, c5, f8, f6, b8, b6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{d8,y} = \begin{cases} \frac{1}{4}, & y = e6, c6, f7, b7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e1,y} = \begin{cases} \frac{1}{4}, & y = f3, d3, g2, c2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e2,y} = \begin{cases} \frac{1}{6}, & y = f4, d4, g3, g1, c3, c1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e3,y} = \begin{cases} \frac{1}{8}, & y = f5, f1, d5, d1, g4, g2, c4, c2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e4,y} = \begin{cases} \frac{1}{8}, & y = f6, f2, d6, d2, g5, g3, c5, c3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e5,y} = \begin{cases} \frac{1}{8}, & y = f7, f3, d7, d3, g6, g4, c6, c4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e6,y} = \begin{cases} \frac{1}{8}, & y = f8, f4, d8, d4, g7, g5, c7, c5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e7,y} = \begin{cases} \frac{1}{6}, & y = f5, d5, g8, g6, c8, c6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e8,y} = \begin{cases} \frac{1}{4}, & y = f6, d6, g7, c7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f1,y} = \begin{cases} \frac{1}{4}, & y = g3, e3, h2, d2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f2,y} = \begin{cases} \frac{1}{6}, & y = g4, e4, h3, h1, d3, d1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f3,y} = \begin{cases} \frac{1}{8}, & y = g5, g1, e5, e1, h4, h2, d4, d2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f4,y} = \begin{cases} \frac{1}{8}, & y = g6, g2, e6, e2, h5, h3, d5, d3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f5,y} = \begin{cases} \frac{1}{8}, & y = g7, g3, e7, e3, h6, h4, d6, d4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f6,y} = \begin{cases} \frac{1}{8}, & y = g8, g4, e8, e4, h7, h5, d7, d5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f7,y} = \begin{cases} \frac{1}{6}, & y = g5, e5, h8, h6, d8, d6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f8,y} = \begin{cases} \frac{1}{4}, & y = g6, e6, h7, d7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g1,y} = \begin{cases} \frac{1}{3}, & y = h3, f3, e2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g2,y} = \begin{cases} \frac{1}{4}, & y = h4, f4, e3, e1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g3,y} = \begin{cases} \frac{1}{6}, & y = h5, h1, f5, f1, e4, e2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g4,y} = \begin{cases} \frac{1}{6}, & y = h6, h2, f6, f2, e5, e3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g5,y} = \begin{cases} \frac{1}{6}, & y = h7, h3, f7, f3, e6, e4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g6,y} = \begin{cases} \frac{1}{6}, & y = h8, h4, f8, f4, e7, e5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g7,y} = \begin{cases} \frac{1}{4}, & y = h5, f5, e8, e6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g8,y} = \begin{cases} \frac{1}{3}, & y = h6, f6, e7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h1,y} = \begin{cases} \frac{1}{2}, & y = g3, f2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h2,y} = \begin{cases} \frac{1}{3}, & y = g4, f3, f1 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h3,y} = \begin{cases} \frac{1}{4}, & y = g5, g1, f4, f2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h4,y} = \begin{cases} \frac{1}{4}, & y = g6, g2, f5, f3 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h5,y} = \begin{cases} \frac{1}{4}, & y = g7, g3, f6, f4 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h6,y} = \begin{cases} \frac{1}{4}, & y = g8, g4, f7, f5 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h7,y} = \begin{cases} \frac{1}{3}, & y = g5, f8, f6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{h8,y} = \begin{cases} \frac{1}{2}, & y = g6, f7 \\ 0, & \text{caso contrario} \end{cases}$$

d)

Ahora hagamos las específicas.

```
In [12]: # $P_{h8,y}$, $P_{g1,y}$, $P_{b7,y}$, $P_{f7,y}$, $P_{e5,y}$.
```

```
#h = 8, 7*8 + 8 -1 = 63
```

```
#g = 7, 6*8 + 1 -1 = 48
```

```
#b = 2, 1*8 + 7 -1 = 14
```

```
#f = 6, 5*8 + 7 -1 = 46
```

```
#e = 5, 4*8 + 5 -1 = 36
```

```
probabilidad_de_transicion(resultados[63][0], resultados[63][1], resultados[63][2])
```

```
probabilidad_de_transicion(resultados[48][0], resultados[48][1], resultados[48][2])
```

```
probabilidad_de_transicion(resultados[14][0], resultados[14][1], resultados[14][2])
```

```
probabilidad_de_transicion(resultados[46][0], resultados[46][1], resultados[46][2])
```

```
probabilidad_de_transicion(resultados[36][0], resultados[36][1], resultados[36][2])
```

$$P_{h8,y} = \begin{cases} \frac{1}{2}, & y = g6, f7 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{g1,y} = \begin{cases} \frac{1}{3}, & y = h3, f3, e2 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{b7,y} = \begin{cases} \frac{1}{4}, & y = c5, a5, d8, d6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{f7,y} = \begin{cases} \frac{1}{6}, & y = g5, e5, h8, h6, d8, d6 \\ 0, & \text{caso contrario} \end{cases}$$

$$P_{e5,y} = \begin{cases} \frac{1}{8}, & y = f7, f3, d7, d3, g6, g4, c6, c4 \\ 0, & \text{caso contrario} \end{cases}$$

e)

```
In [13]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [14]: matriz_de_transicion = np.zeros((64,64))
for x, N_x, len_N_x in resultados_crudos:
    index_x = (x[0]-1)*8 + x[1] - 1
    for vecino in N_x:
        index_vecino = (vecino[0]-1)*8 + vecino[1] - 1
        matriz_de_transicion[index_x][index_vecino] = 1/len_N_x
```

```
In [15]: filas_x = [item[0] for item in resultados]
columnas_y = [y for item in resultados for y in item[1]]
todos_los_estados = sorted(list(set(filas_x + columnas_y)))

# 2. Crear el DataFrame con ceros
matriz = pd.DataFrame(0.0, index=todos_los_estados, columns=todos_los_estados)

# 3. Rellenar las probabilidades  $P(x, y) = 1 / |N(x)|$ 
for x, N_x, len_N_x in resultados:
    probabilidad = 1.0 / len_N_x
    for y in N_x:
        if y in matriz.columns:
            matriz.loc[x, y] = probabilidad

# 4. Reemplazar Los ceros por NaN para que aparezcan en blanco
matriz_limpia = matriz.replace(0.0, np.nan)

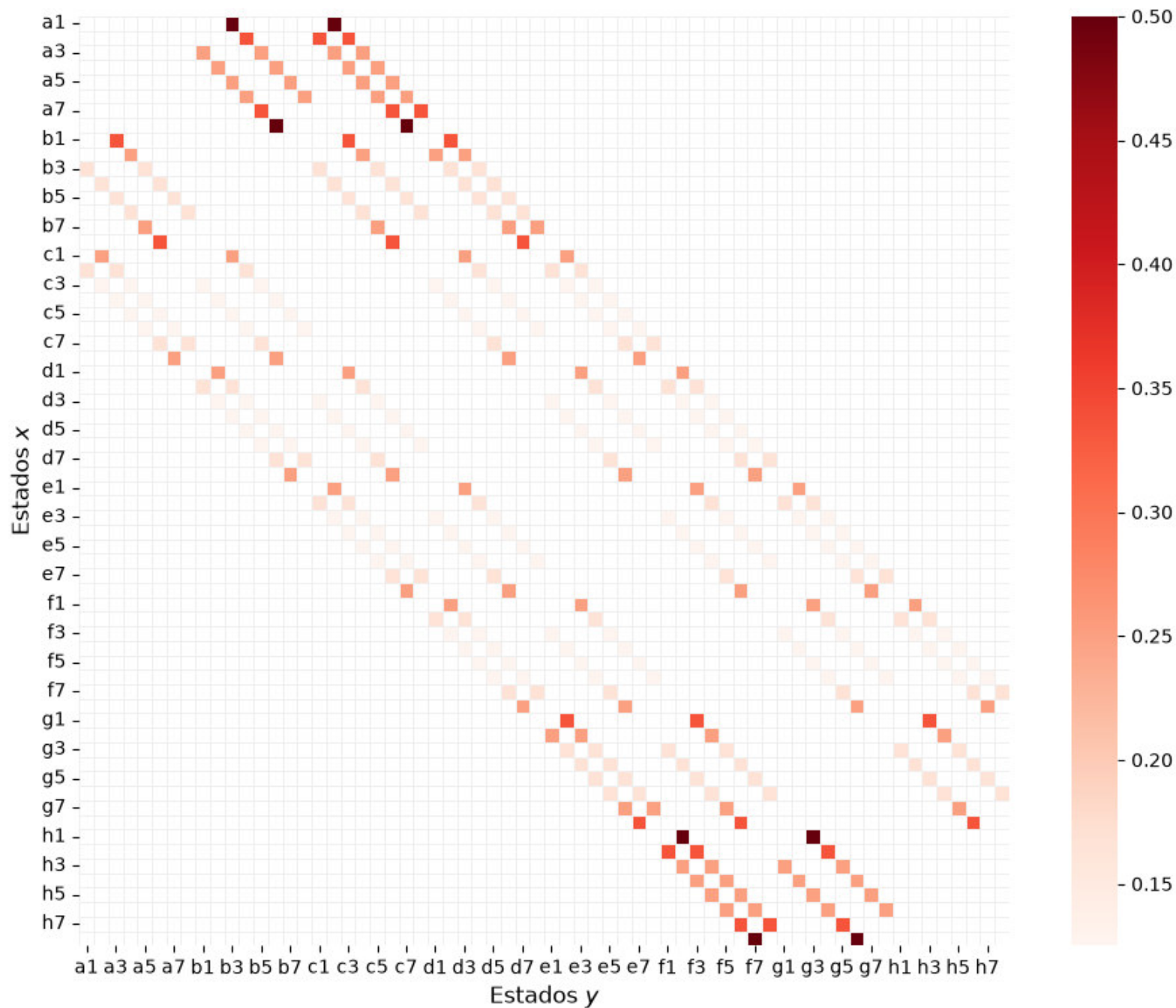
# 5. Configurar y graficar el mapa de calor
plt.figure(figsize=(10, 8))
sns.heatmap(
    matriz_limpia,
    annot=False, # Sin números dentro de las celdas
    cmap="Reds", # Paleta de tonos rojos
```

```
cbar=True, # Barra de color lateral
square=True, # Celdas cuadradas
linewidths=0.5, # Líneas divisorias para notar la cuadrícula
linecolor="#f0f0f0", # Líneas sutiles
mask=matriz_limpia.isnull(), # Fuerza a que los NaN (ceros) se queden blancos
)

plt.title(
    "Mapa de Calor - Matriz de Transición (Tonos Rojos)", fontsize=14, pad=15
)
plt.xlabel("Estados $y$", fontsize=12)
plt.ylabel("Estados $x$", fontsize=12)
plt.xticks(rotation=0)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```

Mapa de Calor - Matriz de Transición (Tonos Rojos)



Ahora veamos la matriz de transición con los porcentajes correspondientes.

```
In [16]: def pos_str(p):
          return f"{'abcdefgh'[p[0]-1]}{p[1]}" if isinstance(p, tuple) else str(p)
```

```
datos = resultados_crudos

estados = [pos_str(item[0]) for item in datos]
m_num = pd.DataFrame(0.0, index=estados, columns=estados)
m_txt = pd.DataFrame("", index=estados, columns=estados)

for x, N_x, n_len in datos:
    origen = pos_str(x)
    for y in N_x:
        destino = pos_str(y)
        m_txt.loc[origen, destino] = f"1/{n_len}"
        m_num.loc[origen, destino] = 1 / n_len

plt.figure(figsize=(15, 13))
sns.heatmap(
    m_num,
    annot=m_txt,
    fmt="",
    cmap=["white"],
    cbar=False,
    square=True,
    linewidths=0.5,
    linecolor="#bbb",
    annot_kws={"size": 6.5},
)

plt.xticks(rotation=90, fontsize=8)
plt.yticks(rotation=0, fontsize=8)
plt.tight_layout()
plt.show()
```

[illegible]

Calculamos la probabilidad de transición $P_{xc2} = \frac{1}{|N(x)|}$ para cada una de estas casillas de origen:

- $|N(a1)| = 2 \implies P_{a1,c2} = \frac{1}{2}$
- $|N(a3)| = 4 \implies P_{a3,c2} = \frac{1}{4}$
- $|N(b4)| = 6 \implies P_{b4,c2} = \frac{1}{6}$
- $|N(d4)| = 8 \implies P_{d4,c2} = \frac{1}{8}$
- $|N(e1)| = 4 \implies P_{e1,c2} = \frac{1}{4}$
- $|N(e3)| = 8 \implies P_{e3,c2} = \frac{1}{8}$

Sumamos las probabilidades condicionales correspondientes:

$$\sum_{x \in N(c2)} P_{xc2} = \frac{1}{2} + \frac{1}{4} + \frac{1}{6} + \frac{1}{8} + \frac{1}{4} + \frac{1}{8} = \frac{17}{12}$$

Sustituyendo en la fórmula de la probabilidad total:

$$P(X_1 = c2) = \frac{1}{64} \cdot \frac{17}{12} = \frac{17}{768}$$

2. $P(X_1 = c2, X_2 = d4, X_3 = e6, X_4 = c7)$

Por la **propiedad de Markov**, multiplicamos la probabilidad del primer estado por la secuencia de probabilidades de transición consecutivas:

$$P(X_1 = c2, X_2 = d4, X_3 = e6, X_4 = c7) = P(X_1 = c2) \cdot P_{c2,d4} \cdot P_{d4,e6} \cdot P_{e6,c7}$$

Evaluando cada término según la cantidad de movimientos $|N(x)|$:

- **Probabilidad inicial en c2:** $P(X_1 = c2) = \frac{17}{768}$ por el punto anterior
- $|N(c2)| = 6 \implies P_{c2,d4} = \frac{1}{6}$
- $|N(d4)| = 8 \implies P_{d4,e6} = \frac{1}{8}$
- $|N(e6)| = 8 \implies P_{e6,c7} = \frac{1}{8}$

Sustituyendo en la fórmula:

$$P(X_1 = c2, X_2 = d4, X_3 = e6, X_4 = c7) = \frac{17}{768} \cdot \frac{1}{6} \cdot \frac{1}{8} \cdot \frac{1}{8} = \frac{17}{294912}$$

3. $P(X_3 = e2, X_2 = d4 \mid X_1 = b3)$

Esto simplifica las cuentas dado que estamos fijando X_1 , por ley de probabilidad condicional

$$P(X_3 = e2, X_2 = d4 \mid X_1 = b3) = P(X_3 = e2 \mid X_2 = d4, X_1 = b3) \cdot P(X_2 = d4 \mid X_1 = b3)$$

Por la **propiedad de Markov** (la transición a X_3 depende únicamente de X_2 y no de X_1):

$$P(X_3 = e2 \mid X_2 = d4, X_1 = b3) = P(X_3 = e2 \mid X_2 = d4) = P_{d4,e2}$$

Por lo tanto, solo debemos calcular $P_{d4,e2}$:

- $|N(b3)| = 6 \implies P_{b3,d4} = \frac{1}{6}$
- $|N(d4)| = 8 \implies P_{d4,e2} = \frac{1}{8}$

$$P(X_3 = e2, X_2 = d4 \mid X_1 = b3) = \frac{1}{6} \cdot \frac{1}{8} = \frac{1}{48}$$

4. $P(X_{n+2} = e3 \mid X_n = a3)$

Esta probabilidad representa la transición en dos pasos de $a3$ a $e3$ podemos hacer el estado intermedio como $k = X_{n+1}$:

$$P(X_{n+2} = e3 \mid X_n = a3) = \sum_{k \in S} P(X_{n+1} = k \mid X_n = a3) \cdot P(X_{n+2} = e3 \mid X_{n+1} = k) = \sum_{k \in S} P_{a3,k} \cdot P_{k,e3}$$

Primero veamos los posibles estados intermediarios k deben cumplir que sean alcanzables desde $a3$ ($k \in N(a3)$) y que desde ellos se pueda alcanzar $e3$ ($k \in N(e3)$):

- **Vecindario de origen:** $N(a3) = \{b1, b5, c2, c4\}$
- **Vecindario de destino:** $N(e3) = \{c2, c4, d1, d5, f1, f5, g2, g4\}$

La intersección de ambos conjuntos es:

$$K = N(a3) \cap N(e3) = \{c2, c4\}$$

Ahora bien tenemos 2 opciones:

1. Ruta por $k = c2$:

- $P_{a3,c2} = \frac{1}{|N(a3)|} = \frac{1}{4}$
- $P_{c2,e3} = \frac{1}{|N(c2)|} = \frac{1}{6}$
- Probabilidad de ruta: $P_{a3,c2} \cdot P_{c2,e3} = \frac{1}{4} \cdot \frac{1}{6} = \frac{1}{24}$

2. Ruta por $k = c4$:

- $P_{a3,c4} = \frac{1}{|N(a3)|} = \frac{1}{4}$
- $P_{c4,e3} = \frac{1}{|N(c4)|} = \frac{1}{8}$
- Probabilidad de ruta: $P_{a3,c4} \cdot P_{c4,e3} = \frac{1}{4} \cdot \frac{1}{8} = \frac{1}{32}$

Finalmente como son 2 rutas independientes, se hace con la probabilidad total, así pues

$$P(X_{n+2} = e3 \mid X_n = a3) = \frac{1}{24} + \frac{1}{32} = \frac{4}{96} + \frac{3}{96} = \frac{7}{96}$$

3. Tablero

- a) Dibuje un tablero de ajedrez de 8×8 .
- b) Pinte inicialmente la casilla $a1$.
- c) Pinte todas las casillas a las que el caballo puede llegar desde $a1$, **en un movimiento**.
- d) A partir de las casillas obtenidas en el paso anterior, pinte las casillas que pueden ser alcanzadas en un movimiento.
- e) Continúe sucesivamente este procedimiento. Si una casilla ya fue pintada en un paso anterior, no es necesario volver a pintarla.
- f) Continúe el procedimiento hasta que no aparezcan nuevas casillas. ¿Cuántas de las 64 casillas del tablero fueron pintadas? ¿Cuáles no fueron pintadas? ¿Qué permite concluir el procedimiento anterior?
- g) ¿se tendrán resultados similares si el caballo inicia en la casilla $d5$? Justifique su respuesta.

Solución

a) hasta f)

```
In [17]: import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from PIL import Image
from pathlib import Path
import imageio.v2 as imageio
from IPython.display import Video
```

```
In [18]: class ChessBoard:
    def __init__(self, marked_squares=None):
        """
        marked_squares:
        [
            (columna, fila, valor),
            ...
        ]
        """
        self.marked_squares = marked_squares or []
        self.files = "abcdefgh"

    def _validate_square(self, square):
        if not isinstance(square, tuple) or len(square) != 3:
            raise ValueError(
                f"Formato inválido: {square}. "
                "Debe ser (columna, fila, valor)"
```

```

    )

    col, row, _ = square

    if not isinstance(col, int) or not isinstance(row, int):
        raise ValueError(f"Columna y fila deben ser enteros: {square}")

    if not (1 <= col <= 8 and 1 <= row <= 8):
        raise ValueError(f"Casilla fuera del tablero: {square}")

def draw(self, save_path=None):
    fig, ax = plt.subplots(figsize=(8, 8))

    # Dibujar tablero
    for row in range(8):
        for col in range(8):
            color = "white" if (row + col) % 2 == 0 else "gray"

            ax.add_patch(
                Rectangle(
                    (col, row),
                    1,
                    1,
                    facecolor=color,
                    edgecolor="black"
                )
            )

    # Marcar casillas y escribir valor
    for square in self.marked_squares:
        self._validate_square(square)

        col, row, value = square

        x = col - 1
        y = row - 1

        # Marca
        ax.add_patch(
            Rectangle(
                (x, y),
                1,
                1,
                facecolor="red",
                alpha=0.6
            )
        )

    # Valor
    ax.text(
        x + 0.5,
        y + 0.5,
        str(value),

```

```

        ha="center",
        va="center",
        fontsize=14,
        fontweight="bold",
        color="white"
    )

    # Numeración
    ax.set_xticks([i + 0.5 for i in range(8)])
    ax.set_xticklabels(self.files)

    ax.set_yticks([i + 0.5 for i in range(8)])
    ax.set_yticklabels(range(1, 9))

    ax.set_xlim(0, 8)
    ax.set_ylim(0, 8)
    ax.set_aspect("equal")

    if save_path:
        plt.savefig(save_path, bbox_inches="tight")

    plt.show()

```

```

In [19]: def posibles_vecinos(x,y):
    return [(x+1,y+2),(x+1,y-2),(x-1,y+2),(x-1,y-2),(x+2,y+1),(x+2,y-1),(x-2,y+1), (x-2,y-1) ]
def vecinos_validos(x,y):
    return [(i,j) for (i,j) in posibles_vecinos(x,y) if i>=1 and i<=8 and j>=1 and j<=8]

```

```

In [20]: def marcar_movimientos(lista: list[tuple], lista_marca: list[tuple], marca: str):
    new = []
    for x, y in lista:
        vecinos = vecinos_validos(x, y)
        for v in vecinos:
            if v not in lista+new:
                new.append(v)
    new_marca = [(i, j, marca) for i, j in new]
    lista += new
    lista_marca += new_marca
    return lista, lista_marca

```

```

In [21]: def crear_tablero(x, y):
    path = f"./images/{x}_{y}"
    Path(path).mkdir(exist_ok=True)

    marca = 0
    lista = [(x, y)]
    lista_marca = [(x, y, str(marca))]
    board = ChessBoard(lista_marca)
    board.draw(f"{path}/tablero_{marca}.png")

    marca = 1
    m, n = 0, -1

```

```

while n != m:
    n = len(lista)
    lista, lista_marca = marcar_movimientos(lista, lista_marca, str(marca))
    m = len(lista)
    if n != m:
        board = ChessBoard(lista_marca)
        board.draw(f"{path}/tablero_{marca}.png")
        marca += 1

```

In [22]:

```

from PIL import Image
from pathlib import Path
from IPython.display import display, Image as IPIImage

```

```

class GifGenerator:
    def __init__(self, folder_path: str):
        self.folder = Path(folder_path)

    def generate(
        self,
        output_path="tablero.gif",
        duration=2000,
        loop=0
    ):
        image_paths = sorted(
            list(self.folder.glob("*.png")) +
            list(self.folder.glob("*.jpg")) +
            list(self.folder.glob("*.jpeg"))
        )

        if not image_paths:
            raise ValueError("No se encontraron imágenes en la carpeta.")

        images = [
            Image.open(path).convert("RGB")
            for path in image_paths
        ]

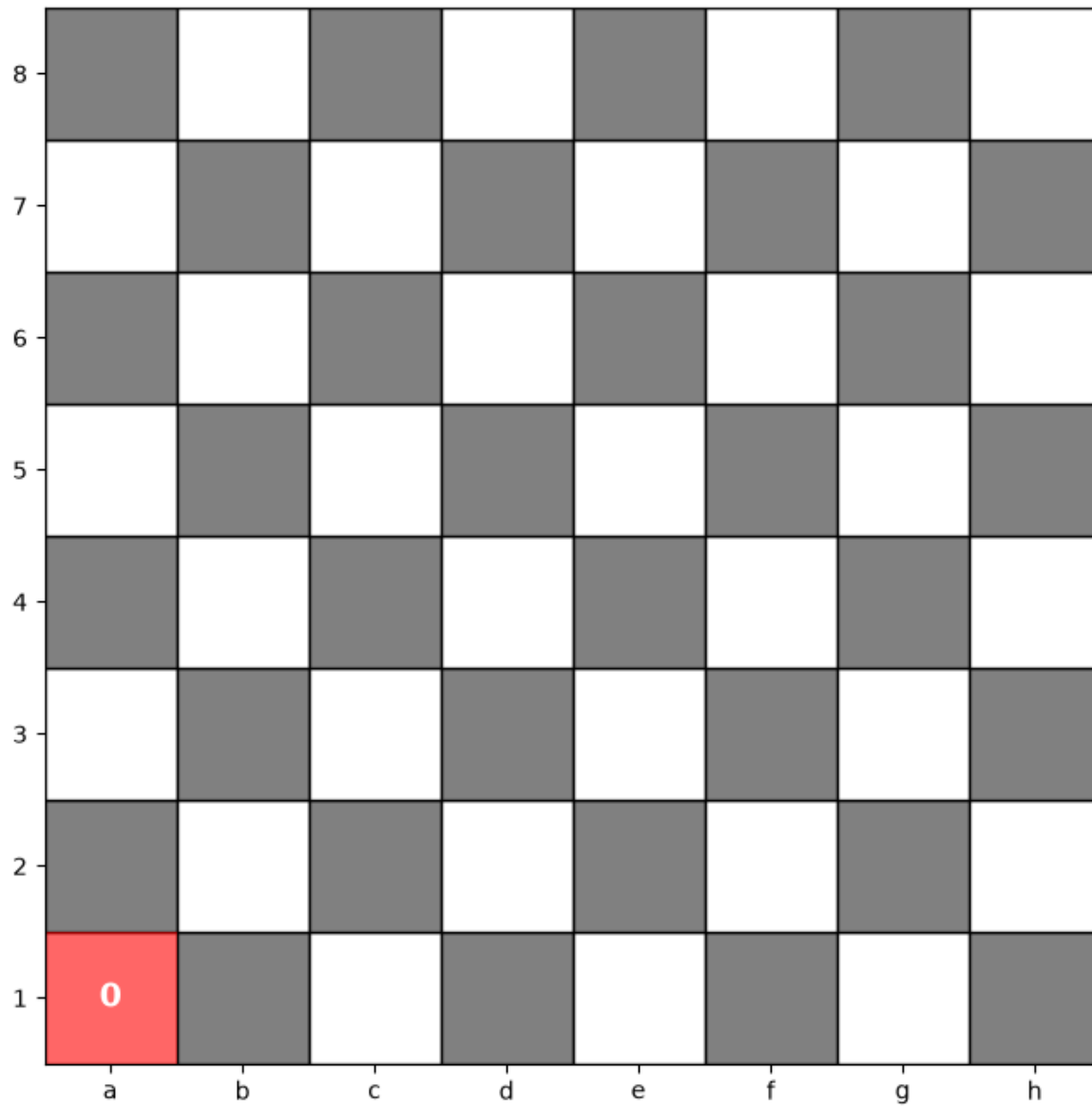
        images[0].save(
            output_path,
            save_all=True,
            append_images=images[1:],
            duration=duration,
            loop=loop
        )

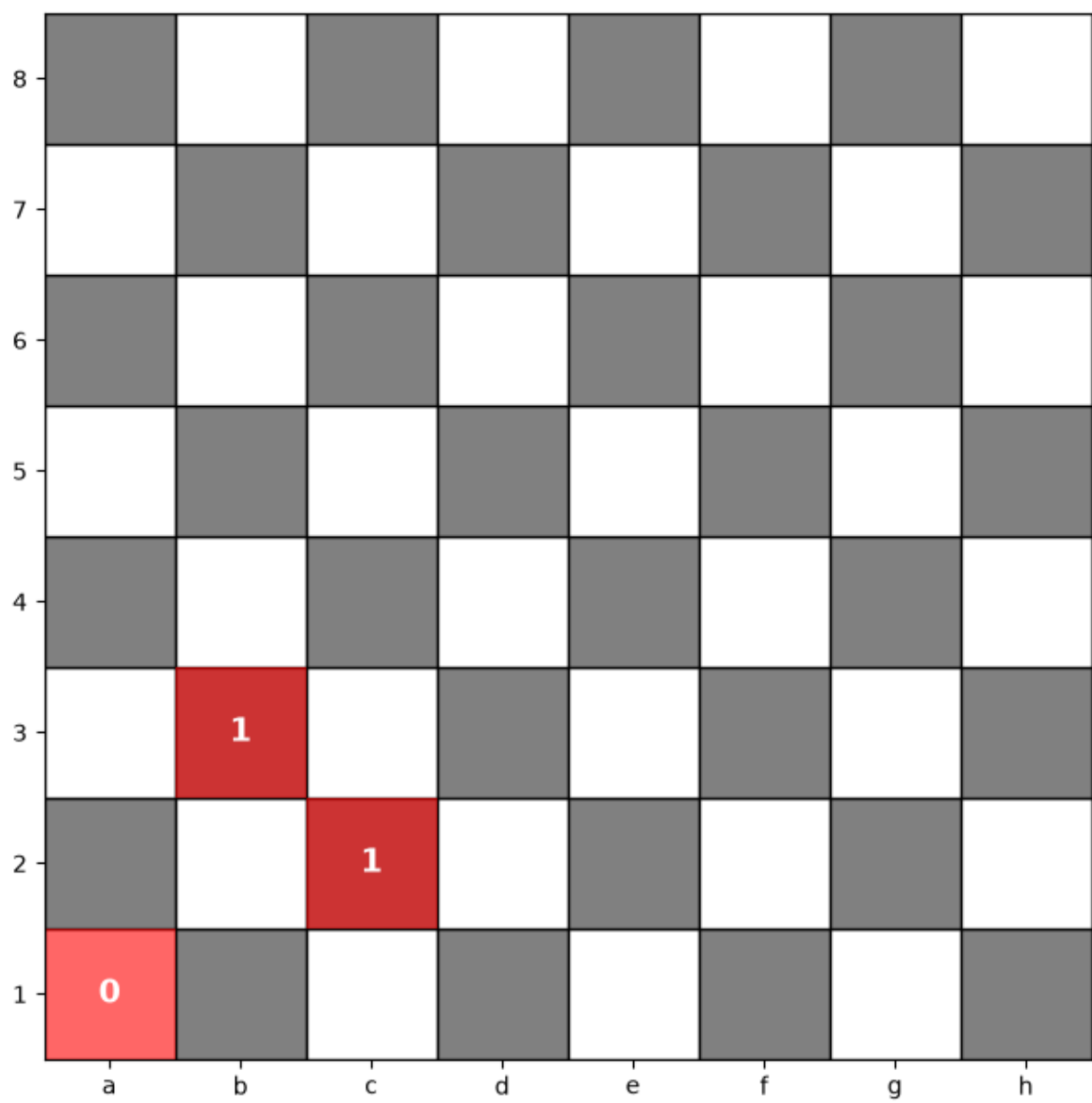
        # Mostrar GIF en el notebook
        display(IPIImage(filename=output_path))

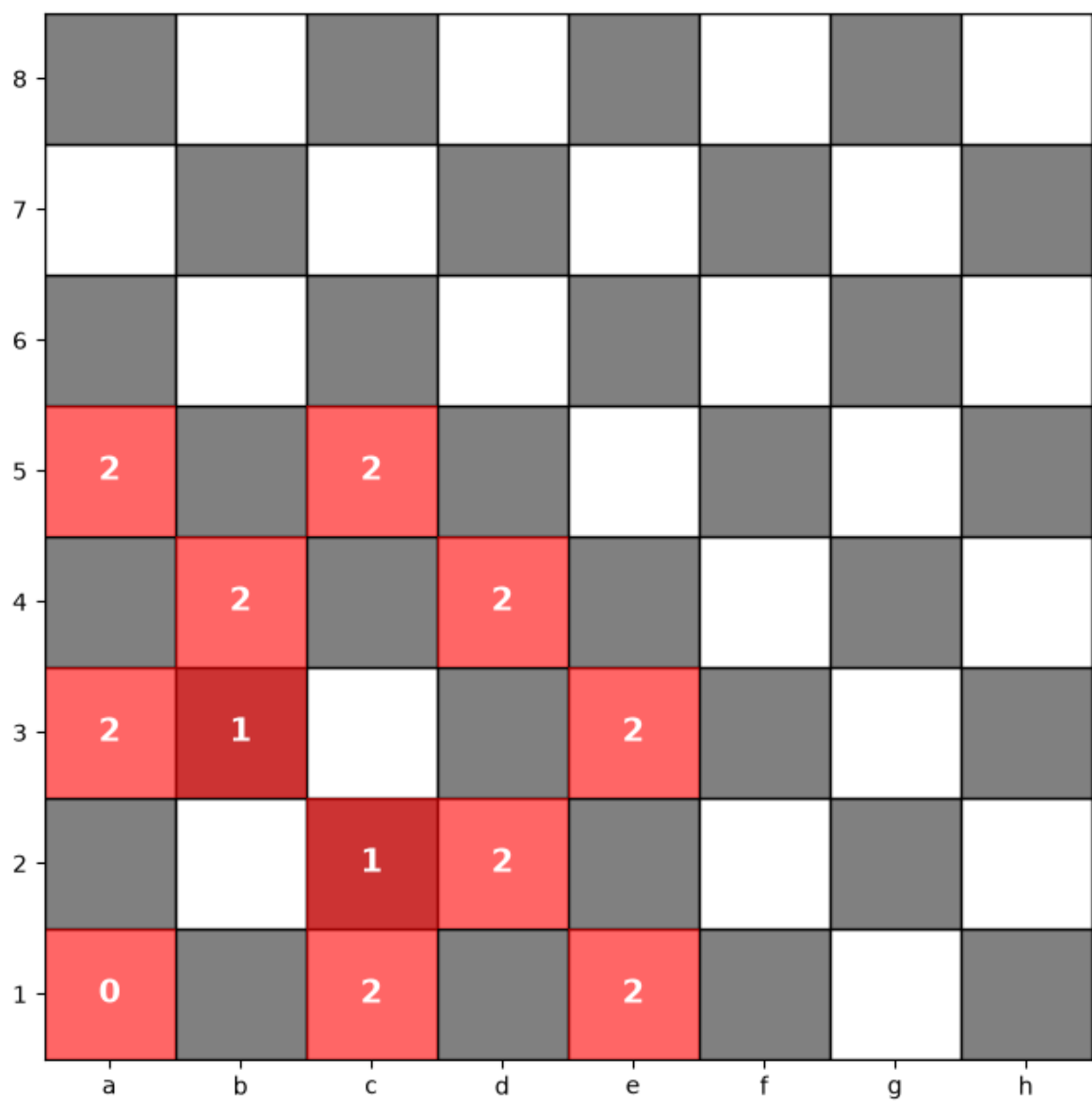
```

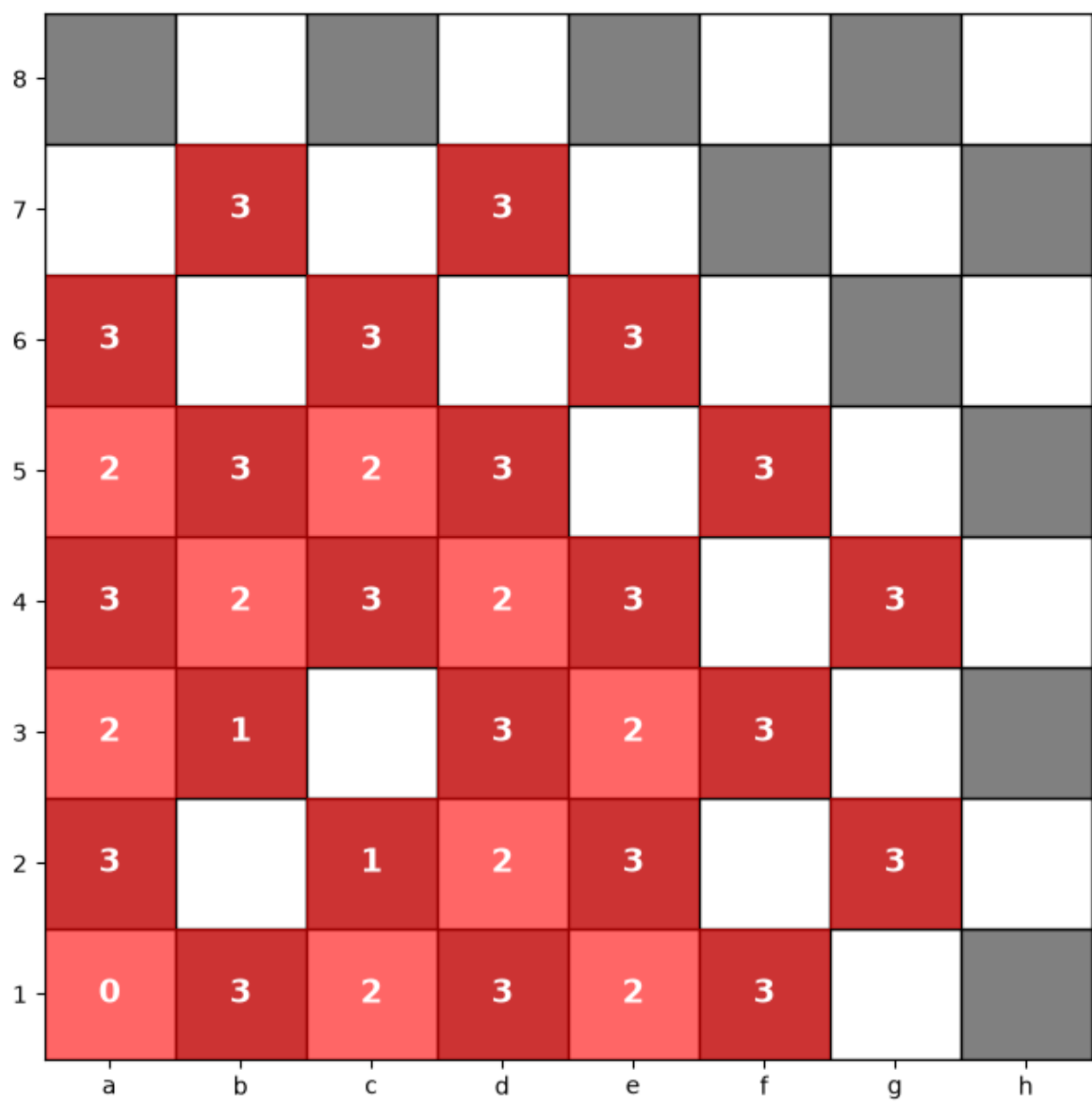
f)

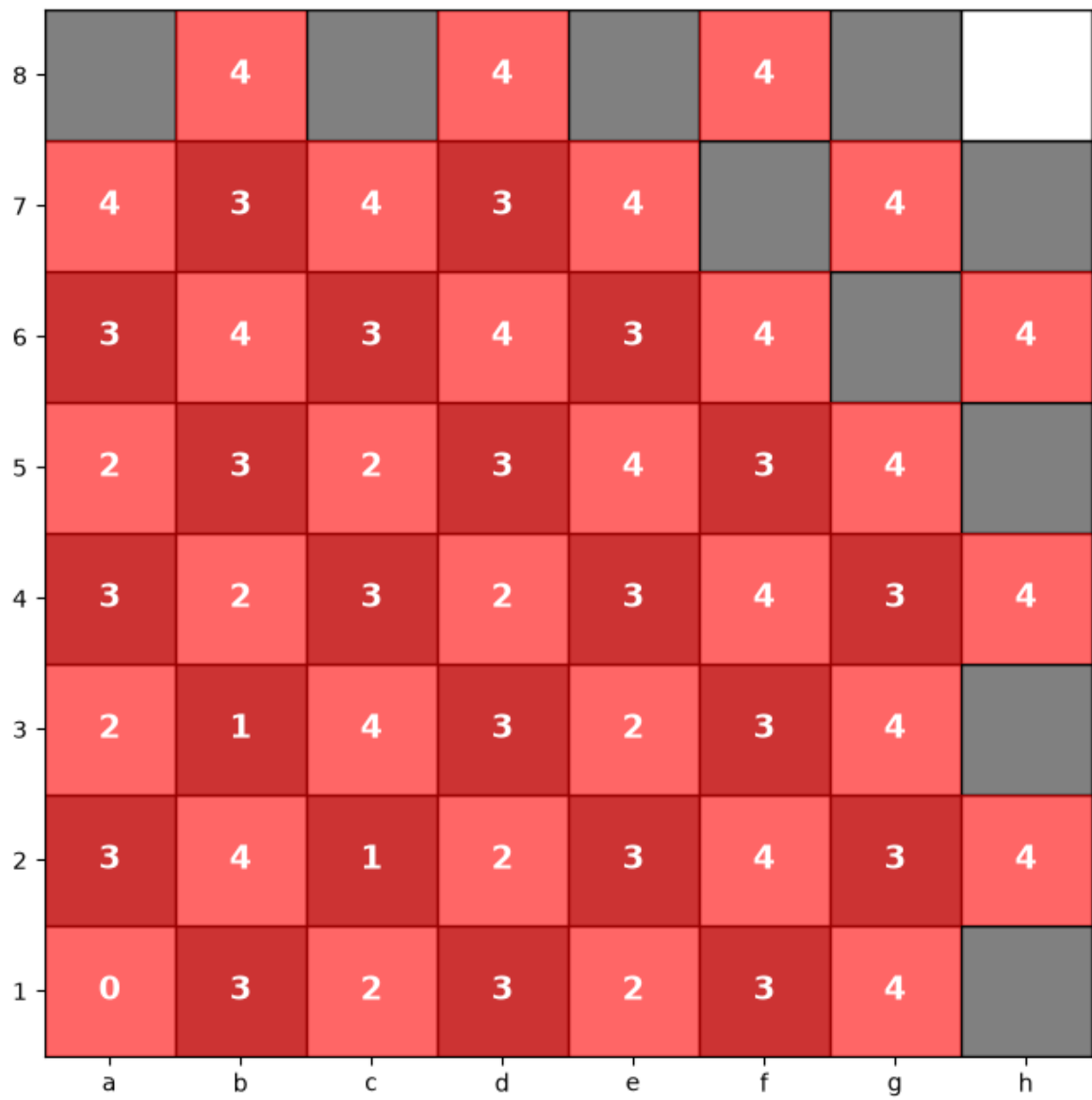
```
In [23]: x, y = 1, 1  
crear_tablero(x, y)
```

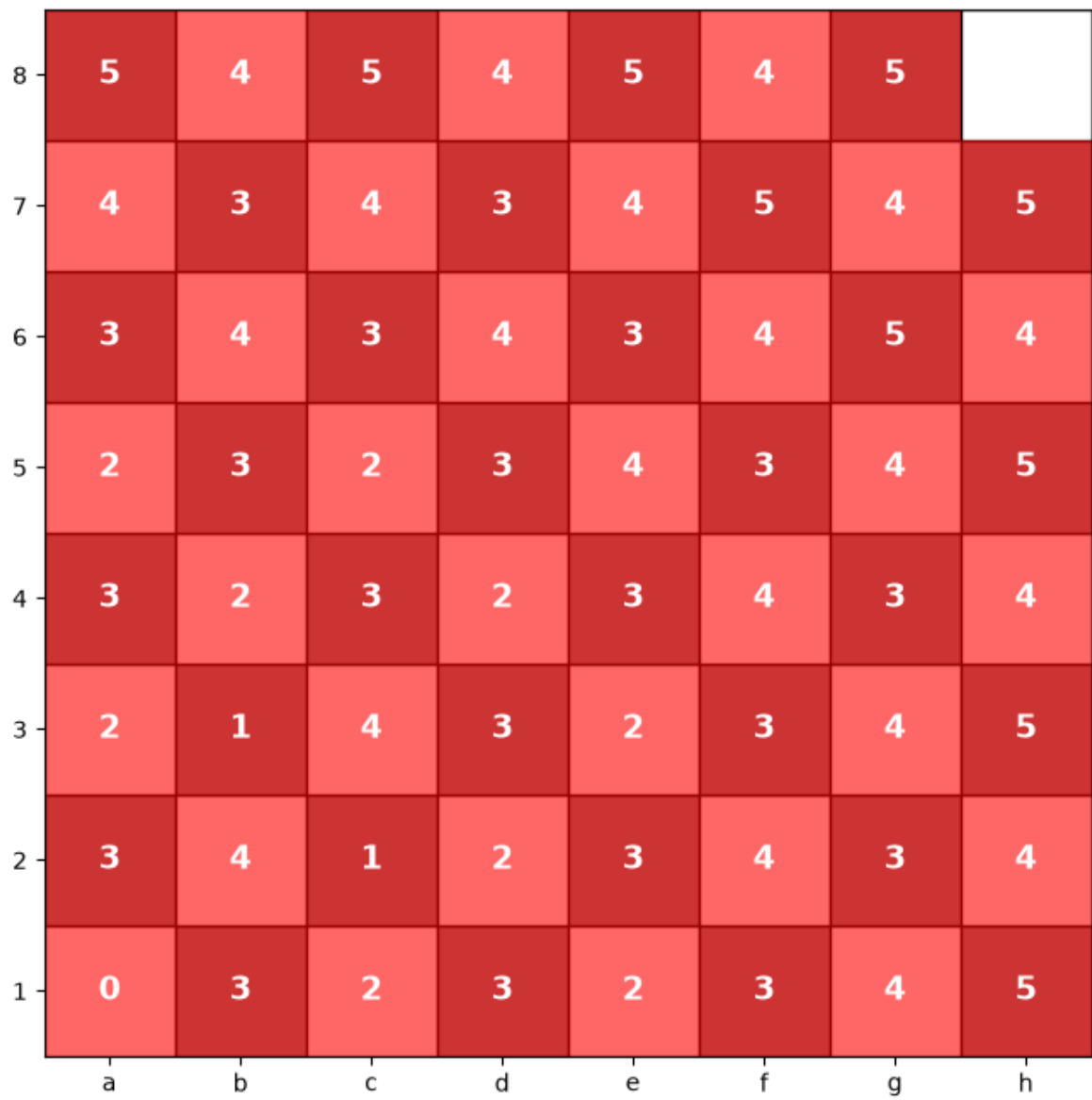


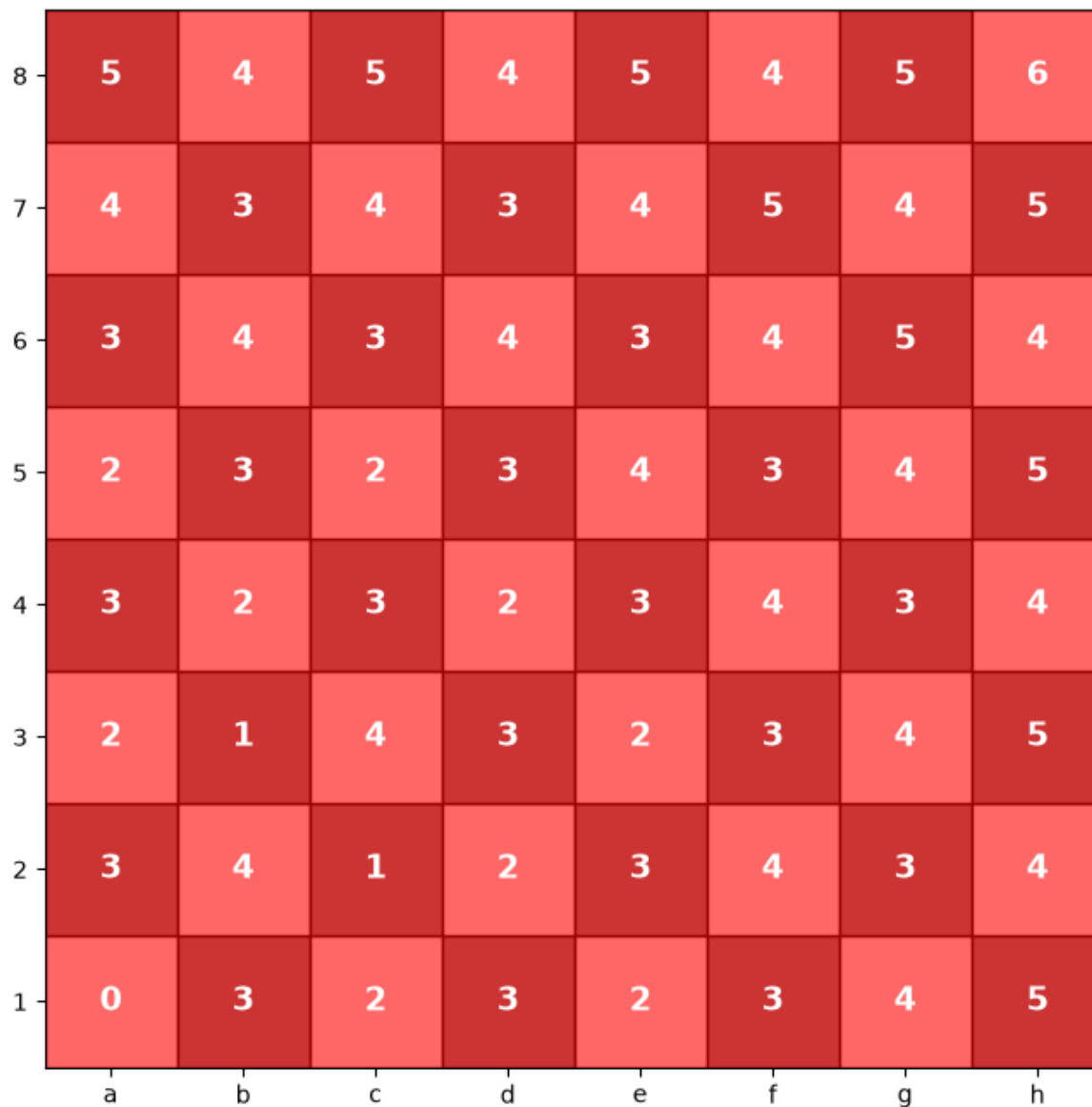












```
In [24]: path = f"./images/{x}_{y}"
generator = GifGenerator(path)

generator.generate(
    output_path=f"./gif/evolucion_tablero_{x}_{y}.gif",
    duration=1500
)
```

<IPython.core.display.Image object>

¿Cuántas de las 64 casillas del tablero fueron pintadas?

- Fueron pintadas todas las 64 casillas.

¿Cuáles no fueron pintadas?

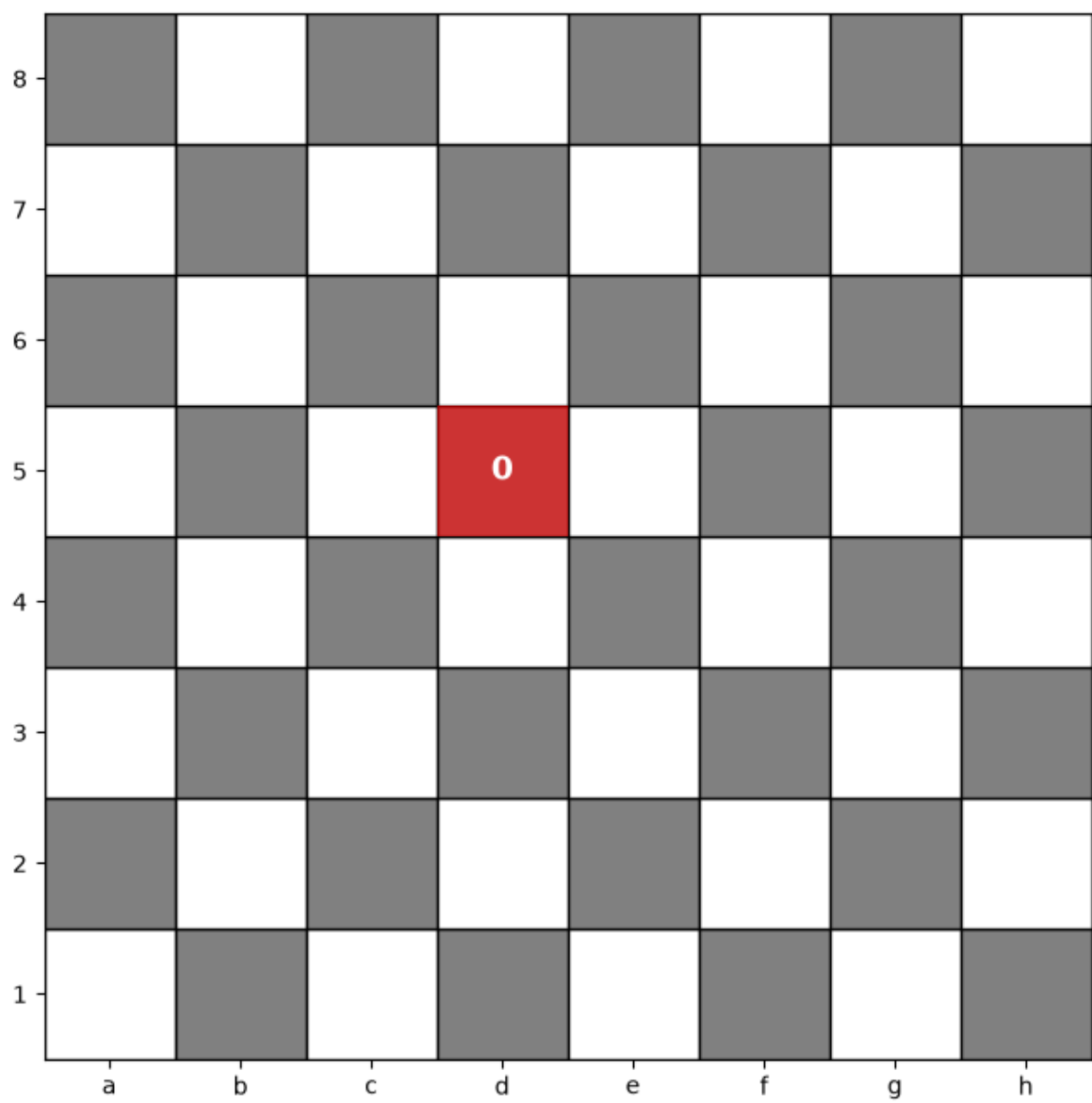
- Ninguna quedó sin pintar

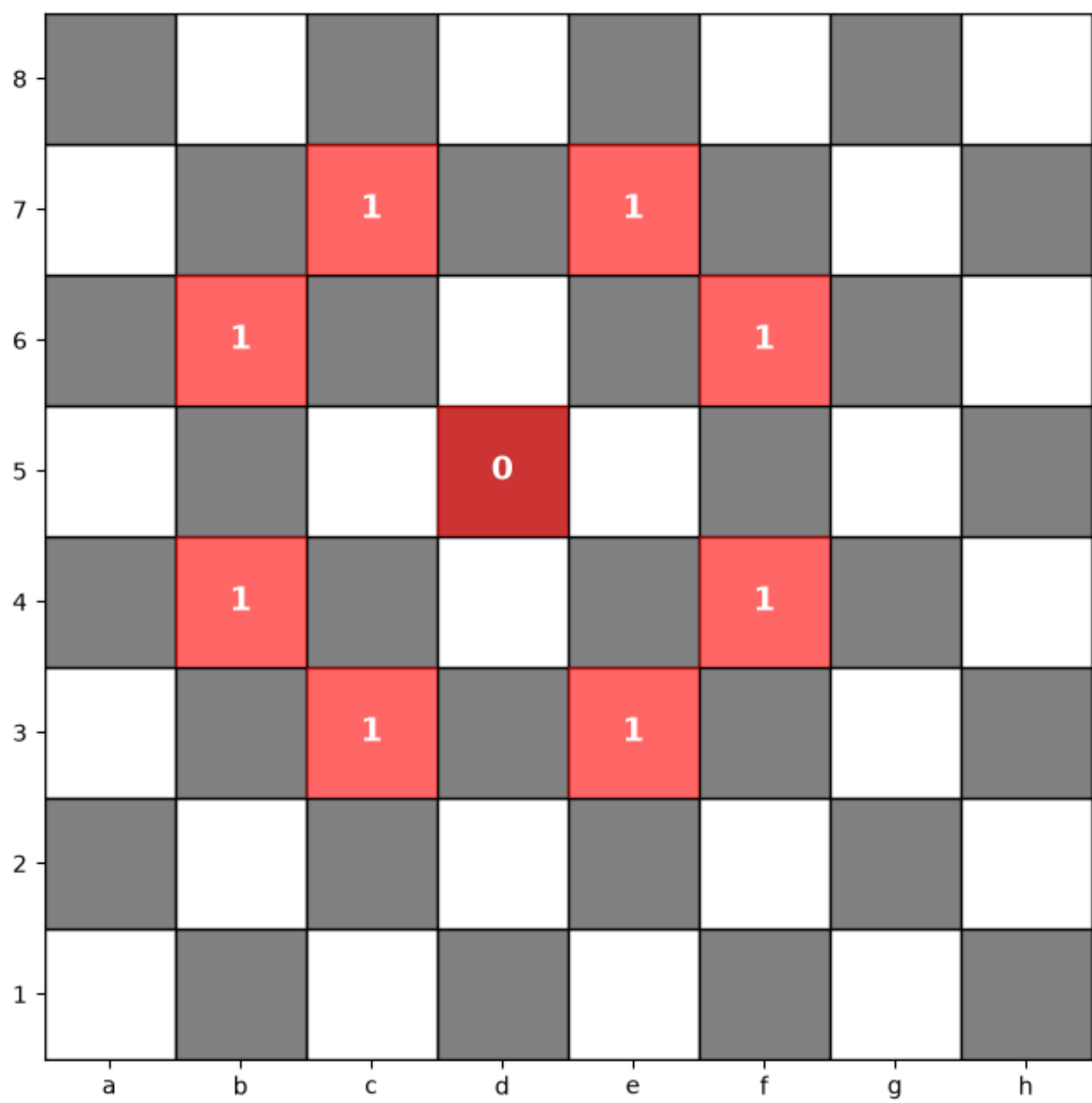
¿Qué permite concluir el procedimiento anterior?

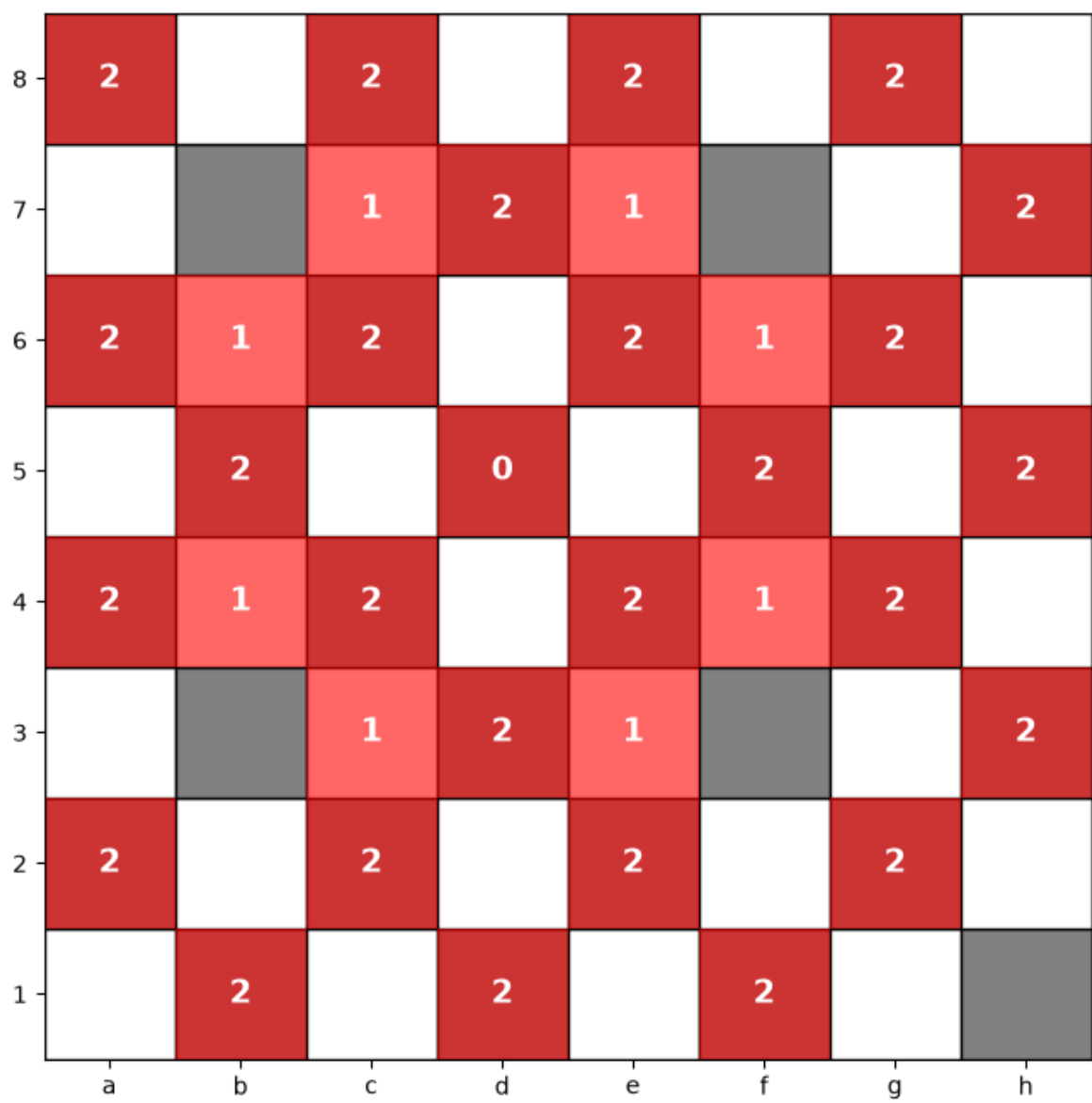
- Que desde $a1$ el caballo puede recorrer todas y cada una de las casillas.

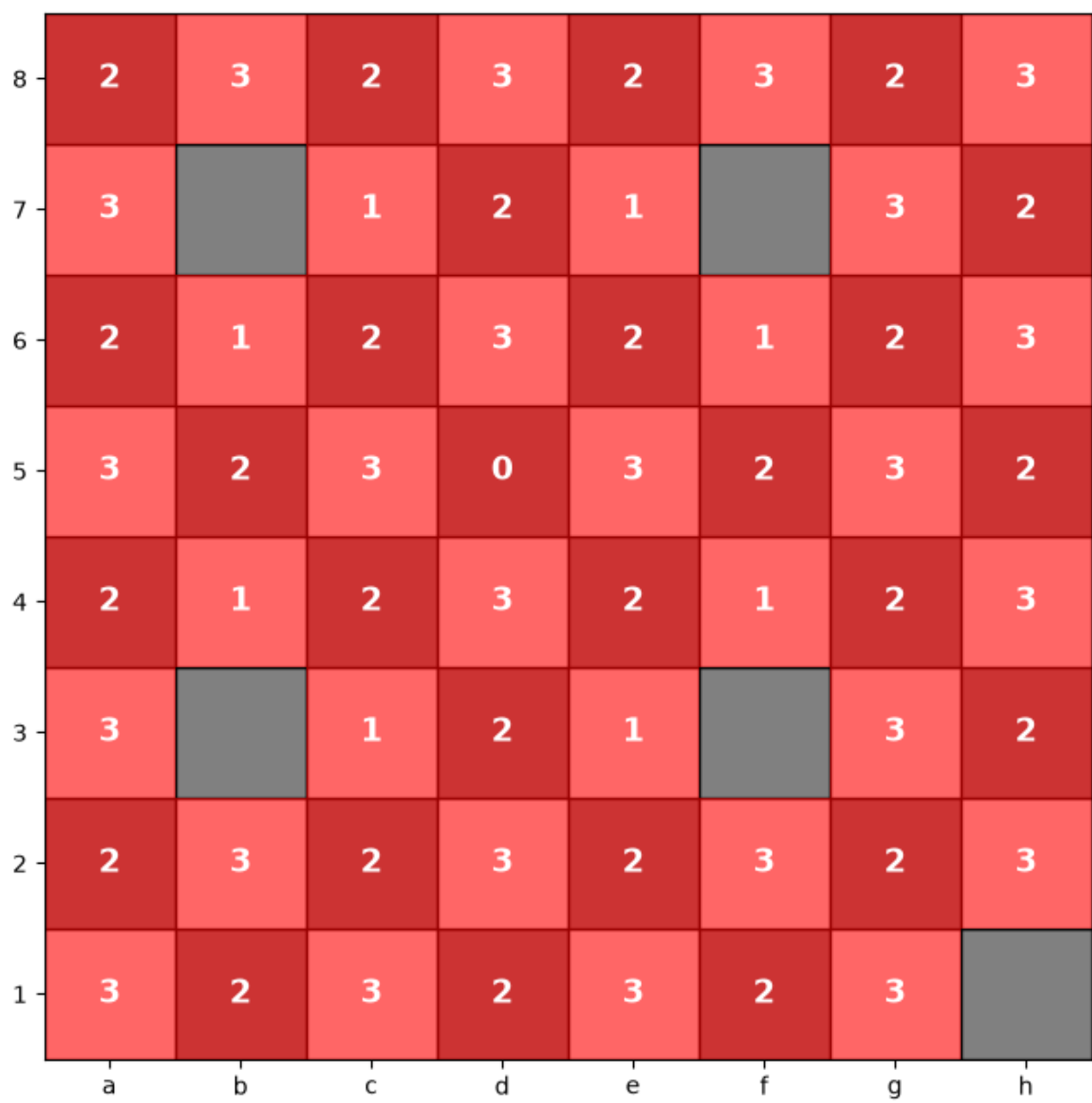
g)

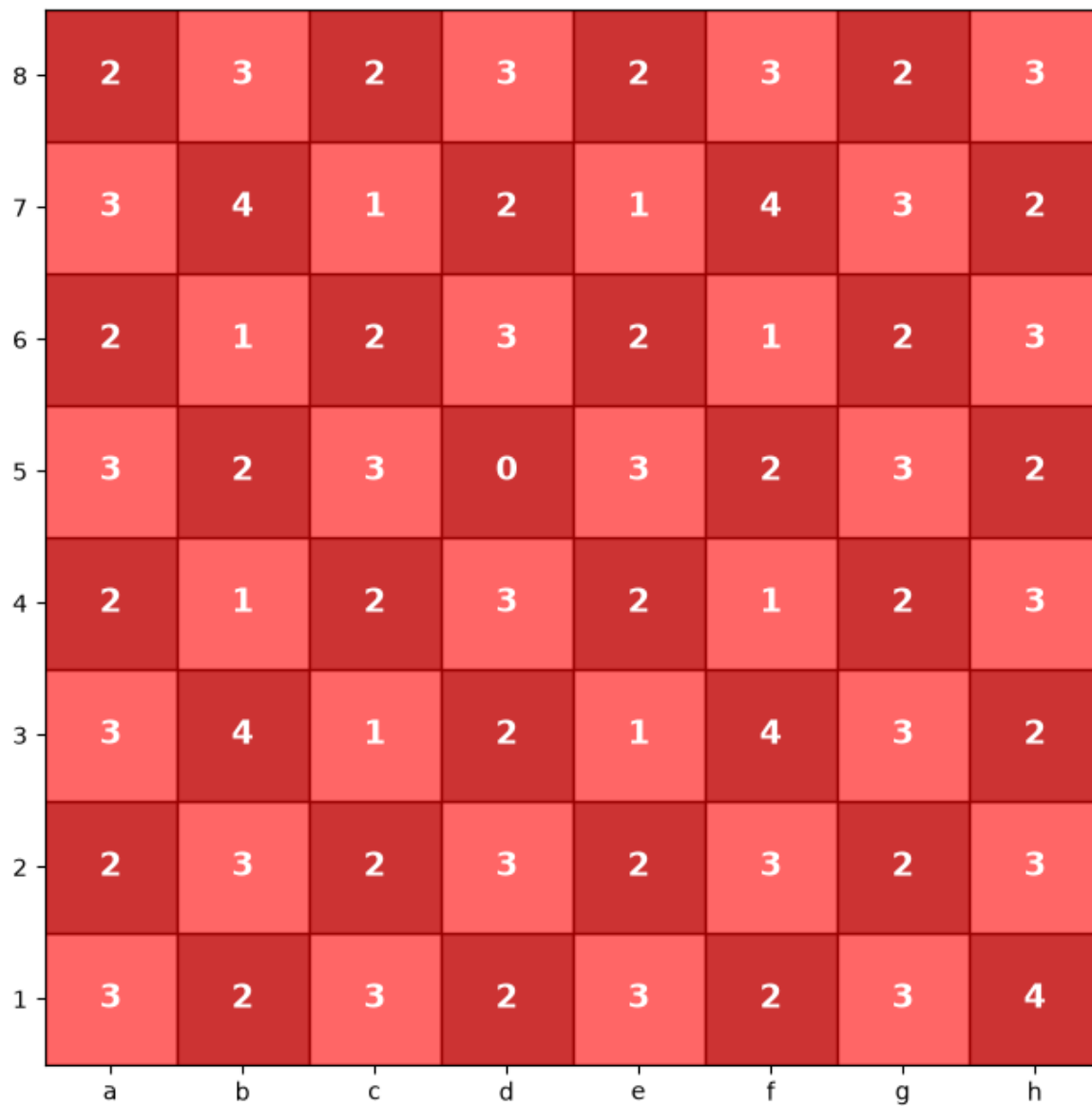
```
In [25]: x, y = 4, 5  
crear_tablero(x, y)
```











```
In [26]: path = f"./images/{x}_{y}"
generator = GifGenerator(path)

generator.generate(
    output_path=f"./gif/evolucion_tablero_{x}_{y}.gif",
    duration=1500
)
```

<IPython.core.display.Image object>

¿Se tendrán resultados similares si el caballo inicia en la casilla $d5$? Justifique su respuesta.

- Se tienen los mismos resultados, se evidenció esto al realizar la ejecución del algoritmo iniciando en la posición $d5$.